IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA (DEEMED TO BE UNIVERSITY)

Yenepoya Institute of

Arts, Science, Commerce

and Management

Project –Low Level Design
on

PREDICTIVE MODELLING FOR HEALTH CARE SUCCESS

Student Name:
Sweety Manoj

Industry Mentor:

Mr.Ujjwal Kumar

Guided by:

Ms.Raksha Adhyanthaya

Table of Contents

1. **Introduction**.....3

1.1. Scope of the document. 3

1.2. Intended Audience..... 3

1.3. System overview..... 4

2. **System Design.** 4

2.1. Application Design.....4

2.2. Process Flow..... 4

2.3. Information Flow.....5

2.4. Components Design 5

2.5. Key Design Considerations 5

2.6. API Catalogue. 6

3. **Data Design**.....6

3.1. Data Model..... 6

3.2. Data Access Mechanism.....6

3.3. Data Retention Policies 6

3.4. Data Migration 7

4. **Interfaces**.....7

5. **State and Session Management**.....7

6. **Caching** 7

7. **Non-Functional Requirements**..... 8

7.1. Security Aspects.....8

7.2. Performance Aspects.....8

8. **References**..... 8

1.Introduction

Stroke is a serious health condition that can lead to disability and death if not detected early. Predicting stroke risk in advance helps in taking timely preventive measures and improving patient care.

This project, *Stroke Prediction using Machine Learning*, aims to analyze patient health data and predict whether a person is at risk of stroke. The system uses a Decision Tree model and provides output as 0 (No Stroke) or 1 (Stroke).

goal of this system is to support early diagnosis and assist in better healthcare decision-making.

1.1 Scope of the Document

This document provides a detailed description of the internal design and implementation of the stroke prediction system. It explains how data is processed, how the machine learning model is trained, and how predictions are generated. It also covers system workflow, modules, and performance considerations.

1.2 Intended Audience

This document is intended for:

- Faculty guide
- Evaluators
- Developers
- Students involved in machine learning projects

1.3 System Overview

The application is designed as a simple and efficient system that includes a user interface, backend processing, and data storage. The frontend allows users to enter data, the backend processes the data and communicates with the prediction model, and the database stores the information for future use.

Input:

Age (continuous numerical value)

Gender (categorical variable)

Hypertension (binary: 0/1)

Heart Disease (binary: 0/1)

BMI (continuous)

Average Glucose Level (continuous)

Smoking Status (categorical)

Output:

- 0 → No Stroke
- 1 → Stroke

The system uses a Decision Tree Classifier to perform binary classification.

2. System Design

2.1 Application Design

The system follows a modular layered architecture ensuring separation of concerns.

Layer Responsibilities

Layer	Description
Presentation Layer	Handles user interaction
Application Layer	Controls workflow
ML Engine Layer	Performs prediction
Data Layer	Manages data

Detailed Workflow Inside Application Layer

1. Accept input from user
2. Validate input data
3. Convert input into structured format
4. Pass data to preprocessing module
5. Invoke ML model for prediction
6. Format output for display

Class-Level Interaction

User → StrokeController → Validator → Preprocessor → ModelService → Output

2.2 Process Flow

Dataset Import
↓
Data Cleaning (handling missing values)
↓
Data Transformation (encoding categorical data)
↓
Train-Test Split
↓
Model Training (Decision Tree)
↓
Model Testing
↓
Prediction
↓
Performance Evaluation

2.3 Information Flow

1. The dataset is first loaded from a CSV file into a pandas DataFrame.
2. The preprocessing module cleans and transforms the data.
3. The processed data is split into training and testing datasets.
4. The model is trained using training data.
5. Predictions are generated using testing and unseen data.
6. The evaluation module calculates accuracy and other metrics.

2.4 Components Design

Input Module:

- Accepts dataset and user input

Preprocessing Module:

- Handles missing values
- Encodes categorical features
- Normalizes data if required

Visualization Module:

- Generates graphs for analysis

Model Module:

- Implements Decision Tree Classifier
- Trains and stores the model

Prediction Module:

- Takes new input data
- Returns classification result

Evaluation Module:

- Calculates accuracy score
- Generates confusion matrix

2.5 Key Design Considerations

- High prediction accuracy
- Simplicity of algorithm
- Fast execution time
- Ease of implementation
- Scalability for future improvements

2.6 API Catalogue

Not applicable. The system operates locally and does not use external APIs.

3. Data Design

3.1 Data Model

The dataset consists of multiple features representing patient health conditions.

Independent Variables:

- age
- bmi
- avg_glucose_level
- hypertension
- heart_disease
- smoking_status

Dependent Variable:

- stroke (target variable)

3.2 Data Access Mechanism

Data is accessed using the pandas library:

```
import pandas as pd
df = pd.read_csv("healthcare-dataset-stroke-data.csv")
```

The dataset is loaded into a DataFrame for further processing.

3.3 Data Retention Policies

- Data is stored locally in CSV format
- No long-term storage or cloud storage is used
- Data is used only during execution

3.4 Data Migration

No data migration is required as the system uses a static dataset without integration from external systems.

4. Interfaces

The system uses Jupyter Notebook as the interface.

Inputs:

- Dataset file
- User-provided patient data

Outputs:

- Prediction result (0 or 1)
- Accuracy score
- Graphical visualizations

5. State and Session Management

- The system operates in a single runtime session
- The model is trained once per execution
- No session persistence or database storage is used

6. Caching

- No caching mechanism is implemented
- All computations are performed dynamically during execution

7. Non-Functional Requirements

7.1 Security Aspects

- Data is processed locally
- No external transmission of sensitive data
- Ensures confidentiality of patient information

7.2 Performance Aspects

- Fast execution due to lightweight model
- Quick prediction response time
- Efficient memory utilization

8. References

- Python Documentation
- Scikit-learn Documentation
- Stroke Prediction Dataset